

Análisis de Arquitectura Técnica — Proyecto HeartCoin

Introducción

El presente documento constituye un análisis técnico integral de la arquitectura del proyecto HeartCoin, una aplicación móvil de impacto social construida sobre Flutter (cliente) y Supabase (backend como servicio). El propósito de este análisis es documentar de manera exhaustiva las decisiones arquitectónicas adoptadas, evaluar su solidez técnica y operativa, identificar riesgos y áreas de mejora, y proporcionar una base de referencia útil tanto para la continuidad técnica del proyecto como para su presentación ante equipos de liderazgo, técnicos o stakeholders externos.

HeartCoin combina funcionalidades de red social (publicaciones, comentarios, seguimiento entre usuarios), un sistema de participación cívica (iniciativas de voluntariado, crowdfunding, impacto social y ahorro), una moneda virtual interna (HeartCoin, HC) canjeable por beneficios ofrecidos por empresas aliadas, y, más recientemente, integraciones con servicios externos de identidad (Original Auth) y procesamiento de pagos (Original Pay).

Objetivo

Evaluar de manera crítica y estructurada la arquitectura general del sistema HeartCoin, abarcando su capa de presentación (aplicación móvil), su capa de datos y lógica de negocio (Supabase/PostgreSQL), su modelo de seguridad, y sus integraciones con servicios de terceros, con el fin de determinar su madurez técnica, sostenibilidad operativa y alineación con buenas prácticas de la industria.

Alcance

Este análisis contempla:

- La aplicación cliente desarrollada en Flutter/Dart, incluyendo su organización de pantallas, servicios y widgets.
- El backend implementado sobre Supabase: base de datos PostgreSQL, sistema de autenticación, Row Level Security (RLS), funciones RPC, triggers y Edge Functions (Deno).
- El modelo económico interno basado en la moneda virtual HeartCoin (HC) y su mecanismo de ledger contable.
- Las integraciones con servicios externos: MediaAAS (almacenamiento de archivos), Original Auth (verificación de identidad/correo) y Original Pay (pasarela de pago con HC como método).
- El modelo de seguridad transversal del sistema (autenticación, autorización, manejo de credenciales).

Quedan fuera del alcance de este documento: pruebas de carga o rendimiento cuantitativas, auditoría de infraestructura de terceros (Supabase, MediaAAS, Original Auth, Original Pay) más allá de su superficie de integración, y aspectos de diseño visual/UX no relacionados con la arquitectura técnica.

Contexto General

HeartCoin opera bajo un modelo de tres roles de usuario diferenciados —**Personal**, **Organización** y **Empresa**—, cada uno con su propia tabla de perfil, flujo de registro y conjunto de funcionalidades. Las organizaciones publican iniciativas de impacto social mediante las cuales los usuarios individuales obtienen HC al participar (mediante check-in geolocalizado o votación en campañas). Las empresas, por su parte, publican beneficios y servicios canjeables con HC, cerrando así el ciclo económico interno de la plataforma.

El sistema no cuenta con un backend tradicional operado de forma independiente: **Supabase actúa como backend integral**, proveyendo base de datos relacional, autenticación, capacidades en tiempo real (Realtime) y cómputo serverless (Edge Functions) bajo un mismo proveedor. Esta decisión arquitectónica reduce la superficie de infraestructura a administrar, pero también introduce una dependencia estructural significativa hacia un proveedor único.

Metodología de Análisis

El presente análisis se elaboró mediante:

1. Revisión directa del código fuente de la aplicación cliente (Dart/Flutter) y de las funciones de servidor (TypeScript sobre Deno).
2. Inspección del esquema de base de datos en producción mediante consultas directas al catálogo de PostgreSQL (`pg_policies`, `pg_constraint`, `pg_trigger`, `pg_proc`), incluyendo el conteo y contenido de políticas de seguridad a nivel de fila y funciones `SECURITY DEFINER`.
3. Análisis de la documentación técnica existente del proyecto (`README.md`, `ENDPOINTS.md`, `ORIGINAL_AUTH_INTEGRATION.md`, `ORIGINAL_PAY_INTEGRATION.md`).
4. Verificación funcional activa de los componentes recientemente construidos (integraciones con Original Auth y Original Pay), mediante ejecución de peticiones firmadas reales contra los endpoints desplegados, no únicamente inspección estática de código.

Desarrollo del Análisis

1. Arquitectura general del sistema

HeartCoin sigue un modelo de **cliente enriquecido con backend como servicio (BaaS)**, en el cual la aplicación Flutter se comunica de forma mayoritariamente directa con Supabase a través de su SDK oficial (`supabase_flutter`), sin mediación de un servidor de aplicación propio para las operaciones

estándar (lectura/escritura de datos, autenticación, tiempo real). Las excepciones a este patrón —operaciones que requieren credenciales privilegiadas o lógica que no puede delegarse al cliente— se implementan mediante **Supabase Edge Functions**, funciones serverless sobre Deno que actúan como intermediarias controladas hacia servicios externos o hacia operaciones administrativas internas.

Este diseño es coherente y deliberado: la seguridad del sistema no descansa en ocultar la clave de acceso del cliente (la `anon key` es, por diseño, pública), sino en dos mecanismos complementarios:

- **Row Level Security (RLS)** a nivel de PostgreSQL, que determina fila por fila qué puede leer o escribir cada usuario autenticado.
- **Funciones SECURITY DEFINER y Edge Functions con clave de servicio**, para las operaciones que requieren validar reglas de negocio no expresables mediante RLS puro (geolocalización, ventanas temporales, suficiencia de saldo, credenciales de terceros).

2. Capa de presentación (aplicación Flutter)

La aplicación cliente está organizada en tres directorios principales: `screens/` (pantallas de interfaz), `services/` (lógica de acceso a datos) y `widgets/` (componentes reutilizables). Se observa un patrón consistente de **servicio-singleton** (`ServiceName.instance`) para encapsular el acceso a cada dominio funcional (autenticación, notificaciones, canjes, seguimiento entre usuarios, etc.), lo cual favorece la cohesión y facilita la localización de lógica de negocio del lado del cliente.

El manejo de estado se resuelve mediante `StatefulWidget` estándar de Flutter combinado con `ValueNotifier/ValueListenableBuilder` para preferencias globales (tamaño de texto, ubicación), sin adopción de un framework de gestión de estado de terceros (Provider, Riverpod, Bloc). Esta decisión simplifica la curva de aprendizaje del proyecto, aunque puede generar mayor verbosidad conforme el número de estados compartidos entre pantallas crezca.

3. Capa de datos y lógica de negocio (PostgreSQL / Supabase)

El esquema relacional cubre los dominios funcionales completos de la aplicación: perfiles por rol, contenido social (publicaciones, comentarios, likes, seguimiento, bloqueos), iniciativas y votaciones, el ledger económico de HC, y el catálogo de beneficios/servicios empresariales.

Row Level Security constituye el mecanismo de autorización primario del sistema: se verificaron **47 políticas activas** que dependen de la función `auth.uid()` para determinar el propietario de cada fila. Este volumen refleja una cobertura de seguridad extensa, aplicada de forma sistemática a prácticamente la totalidad de las tablas del esquema.

La **integridad referencial hacia la identidad del usuario** está fuertemente centralizada: se confirmaron **30 relaciones de clave foránea** apuntando a `auth.users(id)`, todas configuradas con `ON DELETE CASCADE`. Esta característica es la que permite que la eliminación de una cuenta de usuario sea una operación simple y completa (una sola eliminación en `auth.users` elimina en cascada la totalidad de los datos asociados), sin requerir lógica de limpieza manual distribuida por el código de la aplicación.

Los **triggers de PostgreSQL** se emplean de forma consistente para mantener invariantes del sistema sin intervención del cliente: sincronización de contadores (`likes_count`, `comments_count`, `votes_count`), generación automática de notificaciones (con respeto a las preferencias de opt-out del destinatario), sincronización de campos denormalizados (`organization_name`, `company_name`) y, de forma crítica, la actualización del saldo de HC.

4. Modelo económico interno (HeartCoin, HC)

El sistema de moneda virtual se implementa mediante un **patrón de ledger contable**: la tabla `hc_transactions` es la única fuente de verdad de los movimientos de saldo, y el campo `hc_balance` en el perfil del usuario **nunca se escribe directamente**, sino que se deriva automáticamente mediante un trigger (`apply_hc_transaction`) que se dispara en cada inserción sobre el ledger. Este patrón es una práctica sólida de ingeniería financiera a pequeña escala: garantiza trazabilidad completa de cada movimiento y permite, en caso de discrepancia, reconstruir el saldo esperado sumando el historial completo de transacciones.

El acceso de escritura al ledger está restringido exclusivamente a funciones `SECURITY DEFINER` y Edge Functions con clave de servicio —no existe política de `INSERT` directa para el rol `authenticated`—, lo cual impide que un cliente modificado pueda acreditarse HC de forma fraudulenta.

5. Integraciones con servicios externos

El proyecto incorpora tres integraciones externas, todas siguiendo un patrón arquitectónico consistente (Edge Function como intermediaria, credenciales exclusivamente del lado del servidor, firma criptográfica de las peticiones):

- **MediaAAS** (almacenamiento de archivos): proxy mediante la función `upload-media`, que firma las peticiones salientes con HMAC utilizando credenciales que existen únicamente como secretos de servidor.
- **Original Auth** (verificación de identidad): integración de tipo *server-to-server* mediante API key y firma HMAC-SHA256, diseñada deliberadamente como un componente acotado —"puente"— que valida el correo electrónico durante el registro sin alterar el sistema de autenticación principal (Supabase Auth se mantiene como fuente de verdad única para sesiones, login y recuperación de contraseña). Esta decisión de diseño, validada explícitamente con la dirección del proyecto, evita una migración de mayor envergadura y preserva la totalidad del modelo de seguridad RLS ya construido.
- **Original Pay** (pasarela de pago): expone dos endpoints propios (`originalpay-balance`, `originalpay-redeem`) que permiten a un tercero consultar saldo y registrar cobros contra el ledger de HC identificando usuarios por correo electrónico, más un endpoint receptor de webhook (`originalpay-webhook`) que procesa notificaciones asíncronas de estado de pago, incluyendo **reembolso automático e idempotente** de HC ante pagos rechazados.

Es relevante señalar que las tres integraciones, pese a compartir el mismo patrón arquitectónico general, presentan variaciones puntuales en su mecanismo de autenticación (HMAC con nonce y ventana temporal para Original Auth y Original Pay salientes/entrantes propios; comparación de credenciales en el

cuerpo de la petición, sin firma criptográfica, para el webhook entrante de Original Pay, por diseño del proveedor externo). Esta heterogeneidad no constituye una debilidad del diseño de HeartCoin, sino una adaptación necesaria a las especificaciones de cada proveedor.

6. Modelo de seguridad transversal

El manejo de credenciales sigue un principio de **mínimo privilegio y separación estricta cliente-servidor**: ninguna clave de API, secreto compartido o credencial de servicio se incorpora al código de la aplicación móvil; todas residen como secretos configurados en el entorno de ejecución de las Edge Functions. La clave de servicio de Supabase (`SUPABASE_SERVICE_ROLE_KEY`) se inyecta automáticamente por la plataforma en el entorno de cada función, evitando su gestión manual.

Problemas Detectados

1. **Ausencia de atomicidad en operaciones de verificación de saldo.** Tanto las funciones internas de canje (`redeem_beneficio`, `redeem_servicio`) como el endpoint externo `originalpay-redeem` verifican el saldo disponible y posteriormente insertan la transacción en pasos no atómicos entre sí (sin bloqueo de fila `FOR UPDATE`). Bajo un volumen de concurrencia suficiente, dos operaciones simultáneas sobre el mismo usuario podrían, en teoría, provocar un sobregiro momentáneo del saldo.
2. **Ausencia de infraestructura de deep linking.** No existe en el proyecto configuración de App Links, Universal Links ni esquema de URL personalizado, lo cual limita la capacidad de implementar flujos que dependan de redirección desde un navegador externo hacia la aplicación (constatado durante el diseño de la recuperación de contraseña y la evaluación de un eventual flujo OAuth con Original Auth).
3. **Verificación de correo no exigida en el inicio de sesión.** El bloqueo por falta de verificación de correo (integración con Original Auth) solo se aplica en el momento inmediato posterior al registro; un usuario que abandone el flujo antes de completar la verificación puede iniciar sesión normalmente en el futuro sin restricción, dado que `AuthService.signIn` no consulta el campo `email_verified`.
4. **Ausencia de suite de pruebas automatizadas.** No se identificó, dentro del alcance revisado, evidencia de pruebas unitarias, de integración o de extremo a extremo, tanto en la aplicación Flutter como en las Edge Functions.
5. **Ausencia de un pipeline de integración/despliegue continuo (CI/CD) documentado.** El despliegue de Edge Functions y migraciones de base de datos se realiza actualmente mediante ejecución manual del CLI de Supabase.
6. **Persistencia parcial de metadatos en integraciones externas.** El endpoint `originalpay-redeem` acepta un campo `description` que actualmente no se persiste en ninguna columna de la base de datos, representando una funcionalidad documentada como incompleta.
7. **Autenticación de webhook entrante sin firma criptográfica.** El webhook de Original Pay se autentica mediante comparación de credenciales en el cuerpo del mensaje, sin mecanismo de firma ni protección explícita contra reenvío (replay) más allá de la verificación de idempotencia implementada

del lado de HeartCoin. Esta limitación corresponde al diseño del proveedor externo y no es subsanable unilateralmente por HeartCoin.

Riesgos e Impactos

- **Riesgo financiero-operativo (bajo-medio):** la condición de carrera descrita en el punto 1 podría, en escenarios de alta concurrencia (actualmente no observados dado el volumen del proyecto), permitir que un usuario canjee más HC del disponible en su saldo. El impacto se acota naturalmente al ecosistema cerrado de la moneda virtual, sin exposición a pérdida monetaria real directa, salvo en la porción de transacciones que involucran a Original Pay como pasarela de valor real.
- **Riesgo de continuidad operativa (medio):** la arquitectura depende de la disponibilidad simultánea de cuatro proveedores externos (Supabase, MediaAAS, Original Auth, Original Pay). No se documentó un mecanismo de degradación controlada ante la caída de alguno de ellos.
- **Riesgo de calidad y regresión (medio-alto):** la ausencia de pruebas automatizadas incrementa la probabilidad de introducir regresiones no detectadas conforme el sistema crece en complejidad, particularmente relevante dado el ritmo de incorporación de nuevas integraciones observado durante el periodo analizado.
- **Riesgo de seguridad por dispersión de credenciales (medio):** se identificó que credenciales sensibles (API keys de integraciones externas) han sido transmitidas en múltiples ocasiones a través de canales de mensajería no diseñados para el manejo de secretos, lo cual, si bien no compromete la arquitectura del sistema en sí, representa una práctica operativa de riesgo que amerita corrección mediante un gestor de secretos formal.

Fortalezas Identificadas

- **Modelo de seguridad centrado en la base de datos (RLS)** en lugar de depender exclusivamente de la capa de aplicación, lo cual reduce significativamente el riesgo asociado a errores de implementación en el cliente.
- **Patrón de ledger contable** para la moneda virtual, que garantiza trazabilidad y auditabilidad completa de cada movimiento de saldo.
- **Separación disciplinada entre credenciales de cliente y de servidor**, aplicada de forma consistente en las cuatro integraciones externas del sistema.
- **Diseño incremental y conservador de las integraciones de identidad**, particularmente evidenciado en la decisión de acotar Original Auth a un componente auxiliar de verificación en lugar de una migración estructural completa, preservando la inversión ya realizada en el modelo de seguridad existente.
- **Documentación técnica activa y mantenida**, con documentos dedicados por integración (`ORIGINAL_AUTH_INTEGRATION.md`, `ORIGINAL_PAY_INTEGRATION.md`) que registran no solo el diseño sino las incidencias encontradas durante su implementación y verificación.

- **Verificación funcional activa** de los componentes construidos mediante pruebas reales contra los endpoints desplegados, superando la validación puramente estática del código.

Áreas de Mejora

- Incorporar bloqueo de fila (`SELECT ... FOR UPDATE`) o una función transaccional dedicada para las operaciones de descuento de saldo, eliminando la condición de carrera identificada.
- Extender la validación de `email_verified` al flujo de inicio de sesión, si así lo determina una decisión de producto pendiente.
- Establecer una suite de pruebas automatizadas, priorizando las funciones de servidor que manipulan saldo y las políticas de RLS críticas.
- Formalizar un pipeline de CI/CD para el despliegue de Edge Functions y migraciones de base de datos.
- Adoptar un gestor de secretos dedicado para el intercambio de credenciales con terceros, sustituyendo el uso de canales de mensajería informal.
- Completar la persistencia de metadatos actualmente aceptados pero no almacenados (campo `description` en transacciones de Original Pay).
- Evaluar la implementación de infraestructura de deep linking (App Links/Universal Links) como capacidad transversal reutilizable para futuros flujos que la requieran.

Recomendaciones

1. **Priorizar la corrección de la condición de carrera en operaciones de saldo** antes de un incremento significativo en el volumen transaccional, dado que es el hallazgo con mayor impacto potencial identificado en este análisis.
2. **Establecer un umbral mínimo de cobertura de pruebas automatizadas** para la lógica de negocio financiera (ledger de HC, canjes, integraciones de pago), dado que es la superficie de mayor sensibilidad operativa del sistema.
3. **Definir e implementar una política formal de gestión de secretos**, delimitando explícitamente qué credenciales pueden compartirse, con quién y por qué canal, dada la naturaleza multi-proveedor de la arquitectura actual.
4. **Documentar y decidir formalmente la postura de producto respecto a la verificación de correo obligatoria en el login**, actualmente pendiente como decisión no tomada.
5. **Consolidar un pipeline de despliegue automatizado** que reduzca la dependencia de ejecución manual del CLI de Supabase para cambios en producción.

Conclusión

La arquitectura del proyecto HeartCoin evidencia un diseño técnico coherente y deliberado, fundamentado en un uso disciplinado de Row Level Security como mecanismo primario de autorización, un patrón de ledger contable robusto para la gestión de su moneda virtual, y una estrategia de integración con terceros que prioriza consistentemente la separación de credenciales entre cliente y servidor. Las decisiones arquitectónicas más recientes —particularmente el diseño acotado de la integración con Original Auth— reflejan una disciplina de ingeniería orientada a preservar la integridad del sistema existente frente a la presión de incorporar nuevas capacidades.

No obstante, el sistema presenta oportunidades de fortalecimiento concretas y accionables: la atomicidad de las operaciones financieras, la cobertura de pruebas automatizadas, la formalización de un pipeline de despliegue y la gestión estructurada de secretos constituyen las áreas de mayor prioridad para consolidar la madurez operativa del proyecto de cara a un eventual incremento en su escala de usuarios y volumen transaccional.

Anexos

A. Componentes principales del sistema

Componente	Tecnología	Función
Aplicación cliente	Flutter / Dart	Interfaz de usuario multiplataforma (Android/iOS)
Base de datos	PostgreSQL (Supabase)	Persistencia relacional, RLS, triggers, funciones
Autenticación primaria	Supabase Auth	Gestión de sesiones y credenciales de usuario
Cómputo serverless	Supabase Edge Functions (Deno)	Operaciones privilegiadas e integraciones externas
Almacenamiento de archivos	MediaAAS (externo)	Imágenes y documentos de publicaciones/perfiles
Verificación de identidad	Original Auth (externo)	Verificación de correo electrónico en el registro
Procesamiento de pagos	Original Pay (externo)	Consulta de saldo, cobros y reembolsos con HC

B. Documentación técnica de referencia existente en el repositorio

- `README.md` — panorama general del stack, estructura del proyecto y funcionalidades por pantalla.

- `ENDPOINTS.md` — catálogo detallado de endpoints expuestos vía PostgREST, RPC y Edge Functions.
- `ORIGINAL_AUTH_INTEGRATION.md` — diseño e implementación de la integración de verificación de correo.
- `ORIGINAL_PAY_INTEGRATION.md` — diseño e implementación de la integración de pagos con HC.

C. Información adicional recomendada para profundizar futuros análisis

- Métricas cuantitativas de uso real (volumen de usuarios activos, frecuencia de transacciones de HC) para dimensionar con precisión el riesgo de las condiciones de carrera identificadas.
- Registro histórico de incidentes operativos, si existiera, para contrastar contra los riesgos teóricos aquí documentados.
- Política de retención de datos y cumplimiento normativo aplicable a la jurisdicción de operación del proyecto, complementaria al aviso de privacidad ya implementado en la aplicación.